

Lec23

Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that allows us to run JavaScript outside the browser.

Before Node.js:

- JavaScript was mainly used inside browsers.
- It handled webpage interactivity like:
 - Button clicks
 - Form validation
 - Animations

After Node.js:

- JavaScript can run on servers.
- We can build:
 - Web servers
 - APIs
 - Real-time chat apps
 - Command-line tools
 - Streaming apps

Example

```
console.log("Hello from Node.js");
```

This JavaScript runs directly in terminal using Node.js.

Why Node.js Was Created?

Browsers can execute JavaScript, but developers wanted JavaScript for backend development too.

Node.js was introduced in **2009** by Ryan Dahl.

Goals:

- Fast
- Scalable
- Event-driven
- Non-blocking

Why Learn Node.js?

Advantages:

- Same language for frontend + backend (JavaScript)
- Fast execution
- Large ecosystem
- Easy scalability
- Huge community support

Popular companies using Node.js:

- LinkedIn
- Uber

Installing Node.js

Download from:

[Node.js Official Website](https://nodejs.org/)

Check installation:

```
node -v
```

Example output:

```
v20.5.0
```

Check npm:

```
npm -v
```


Understanding Runtime

What is Runtime?

A runtime is an environment where code executes.

Examples:

- Browser runtime → JavaScript runs in browser
- Node runtime → JavaScript runs in Node.js

Node.js Runtime Architecture

Node.js runtime mainly contains:

- V8 Engine
- Event Loop
- APIs
- Memory handling

V8 Engine

V8 is Google's JavaScript engine.

It converts JavaScript into machine code.

Flow:

JavaScript Code → V8 Engine → Machine Code → Execution

Example:

```
let a = 10;
```

```
let b = 20;
```

```
console.log(a + b);
```

V8 executes this quickly.

Event Driven Architecture

Node.js works using events.

Example:

- User clicks button
- Request comes to server
- File finishes reading

Node reacts to events.

Example:

```
button.onclick = function() {  
  console.log("Clicked");  
}
```

Non-Blocking I/O

This is the biggest strength of Node.js.

Traditional blocking:

Read File → Wait → Process

Node.js non-blocking:

Read File → Continue Other Work → File Ready → Process

Example:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Task Done");  
}, 2000);
```

```
console.log("End");
```

NPM (Node Package Manager)

NPM stands for **Node Package Manager**.

It helps install libraries/packages.

Think of npm as:

- App Store for Node.js
- Library manager

Why Use NPM?

Instead of writing everything from scratch, use packages.

Examples:

- Express
- React tools
- Lodash
- Axios

Install package:

```
npm install express
```

Shortcut:

```
npm i express
```

package.json

Important project file.

It stores:

- Project name
- Version
- Dependencies
- Scripts

create:

npm init

Or:

npm init -y

Example:

```
{  
  "name": "demo",  
  "version": "1.0.0",  
  "main": "index.js"  
}
```

Dependencies

When package installed:

```
npm install express
```

package.json updates:

```
"dependencies": {  
  "express": "^4.18.0"  
}
```

node_modules

Contains installed packages.

Example:

project/

├── node_modules/

├── package.json

└── package-lock.json

Useful NPM Commands

Install:

```
npm install package-name
```

Install globally:

```
npm install -g package-name
```

Uninstall:

```
npm uninstall package-name
```

Modules in Node.js

Modules are reusable pieces of code.

They help divide code into smaller files.

Benefits:

- Reusable
- Organized
- Maintainable

Types

Core Modules

Local Modules

Third-party Modules

Core Modules

Built into Node.js.

Examples:

- fs
- http
- path
- os

Example:

```
const os = require("os");
```

```
console.log(os.platform());
```

Local Modules

our own files.

math.js

```
function add(a, b) {  
  return a + b;  
}
```

module.exports = add;

app.js

```
const add = require("./math");
```

```
console.log(add(10, 20));
```


Third-party Modules

Installed via npm.

Example:

```
npm install express
```

Use:

```
const express = require("express");
```

Module System

Two main systems:

CommonJS

Uses:

`require()`

`module.exports`

Example:

```
const data = require("./file");
```

ES Modules

Uses:

import

export

Example:

```
import data from "./file.js";
```

File System Module

One important module is `fs`.

Used for:

- Reading files
- Writing files
- Updating files

Example:

```
const fs = require("fs");
```

```
fs.writeFileSync("test.txt", "Hello Students");
```

Read:

```
const data = fs.readFileSync("test.txt", "utf8");  
console.log(data);
```

Creating Server in Node.js

Using HTTP module:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.write("Hello Students");
  res.end();
});
```

Run:

```
node app.js
```

Visit:

localhost:3000

Server responds with:

Hello Students

Task

Create a Node.js command-line application